Pictor Prestige Camera-Client wireless communication API

Document code	SPC70001873
Document name	R 2.0 Pictor Prestige Camera-Client wireless communication API
Document type	Specification

Contents

1 Purpose of the document

Purpose of this document is to describe the wireless interface between Pictor Prestige -camera and Client Software. This document revision describes interface level 2.

2 Wireless Communication API

In this chapter the API between Client SW and Camera SW is described.

2.1 Use cases

Use cases for the interface between Client Software and Camera are illustrated in Figure 1. Camera provides access point mode and network mode as options for connecting with the device running the Client software. In network mode the Camera must be connected to the same wireless network with the device running the Client software. In access point mode the camera creates a wireless network to which the Client software can connect.

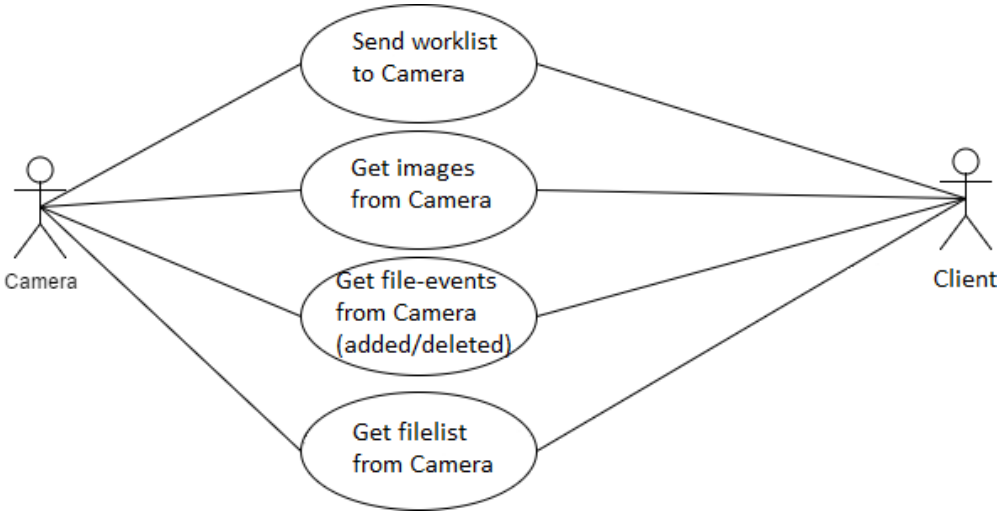

FIGURE 1 USE CASE DIAGRAM OF COMMUNICATION BETWEEN CAMERA AND CLIENT.
NOTE: AUTHENTICATION IS SUPPORTED ONLY IN ACCESS POINT MODE (BY WLAN-PASSWORD SET BY CAMERA).

2.2 Network specification

Specification of network protocols is listed below.

Wi-Fi-protocol:	802.11 b/g/n
Security:	WPA2 is supported. When creating Access Point with Camera, WPA2 is used for security protocol.
TCP/IP, UDP/IP:	Transferred in IPv4 packet.

Camera can be used either in network mode or access point -mode. In network mode, Camera connects to an existing access point. In access point -mode, Camera creates access point to which Client software can connect to. Instructions on how to setup network mode can be found in Pictor Prestige user manual. Sequence diagram for connecting Client to camera in access point -mode can be seen in Figure 2. Sequence diagram for connecting Client to camera in network -mode can be seen in Figure 3. Pairing Camera with Client is described in chapter 2.3.

Camera discovery is done by sending and receiving UDP-packets in the network. Other messages are sent via TCP-socket.

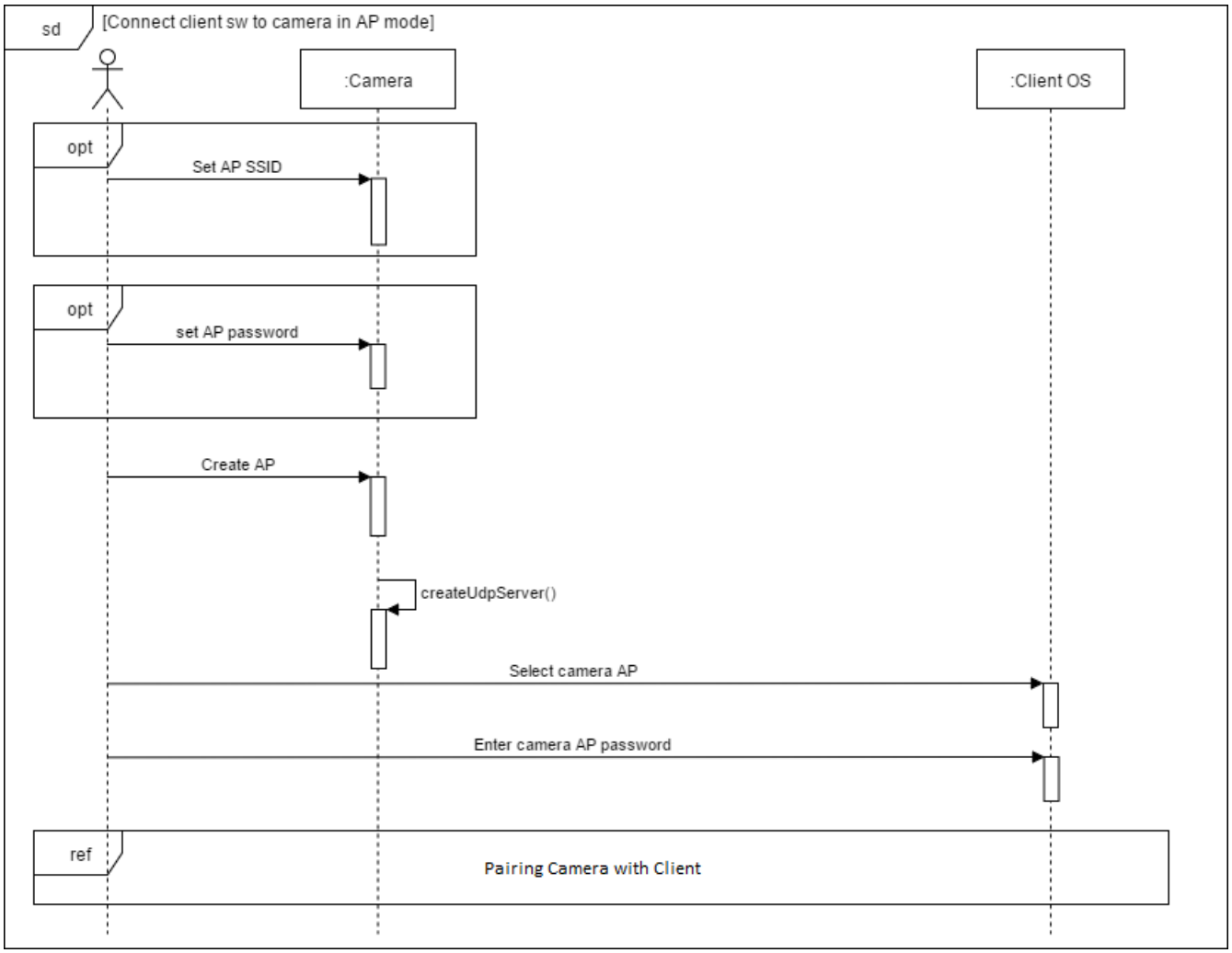

FIGURE 2 CONNECT CLIENT TO CAMERA IN ACCESS POINT MODE

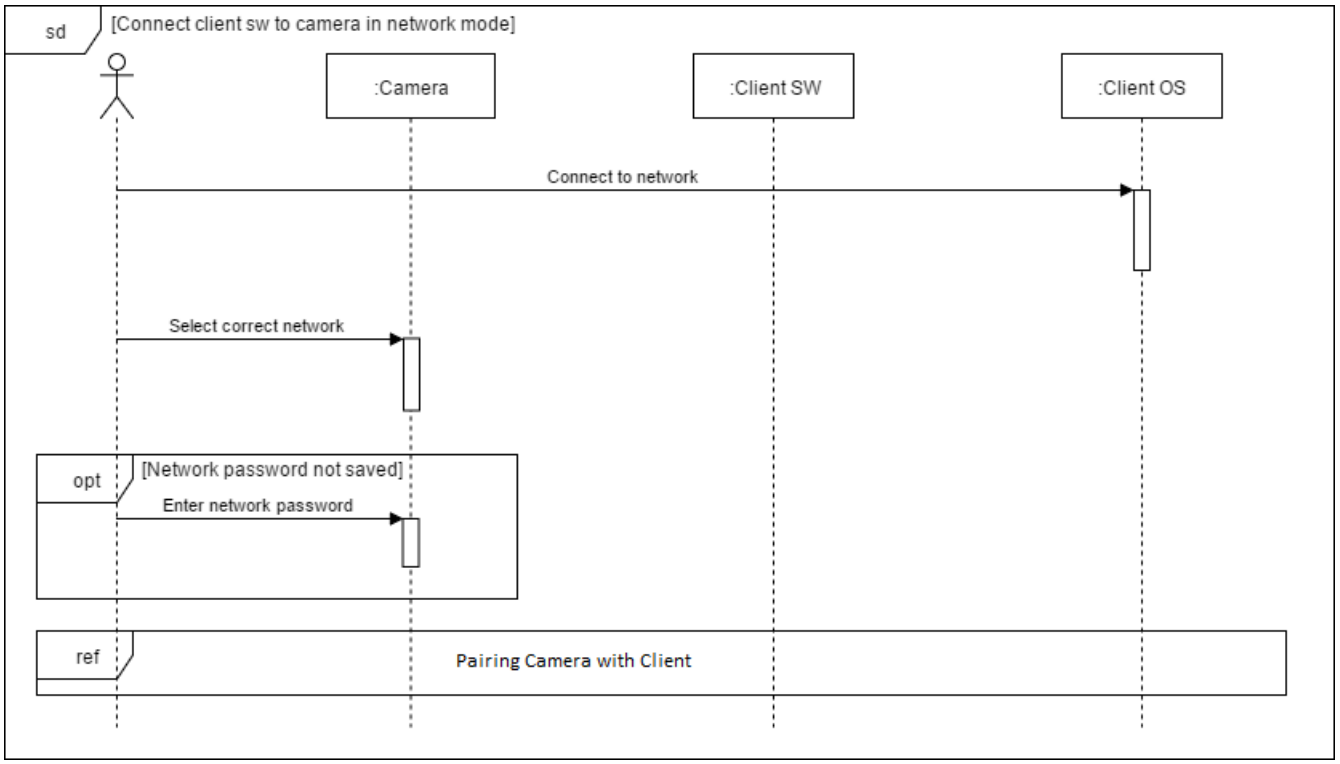

FIGURE 3 CONNECT CLIENT TO CAMERA IN NETWORK MODE

2.3 Camera discovery and pairing Camera with Client

Client software sends a broadcast UDP packet to detect cameras in the network (DETECT_CAMERA, Table 1). Camera is listening on UDP-port 3000. The Camera response (UDP packet CAMERA_DETECTED, Table 1) is sent to the IP address where the detect camera packet was received from. Sequence diagram of camera discovery and pairing is shown in Figure 4.

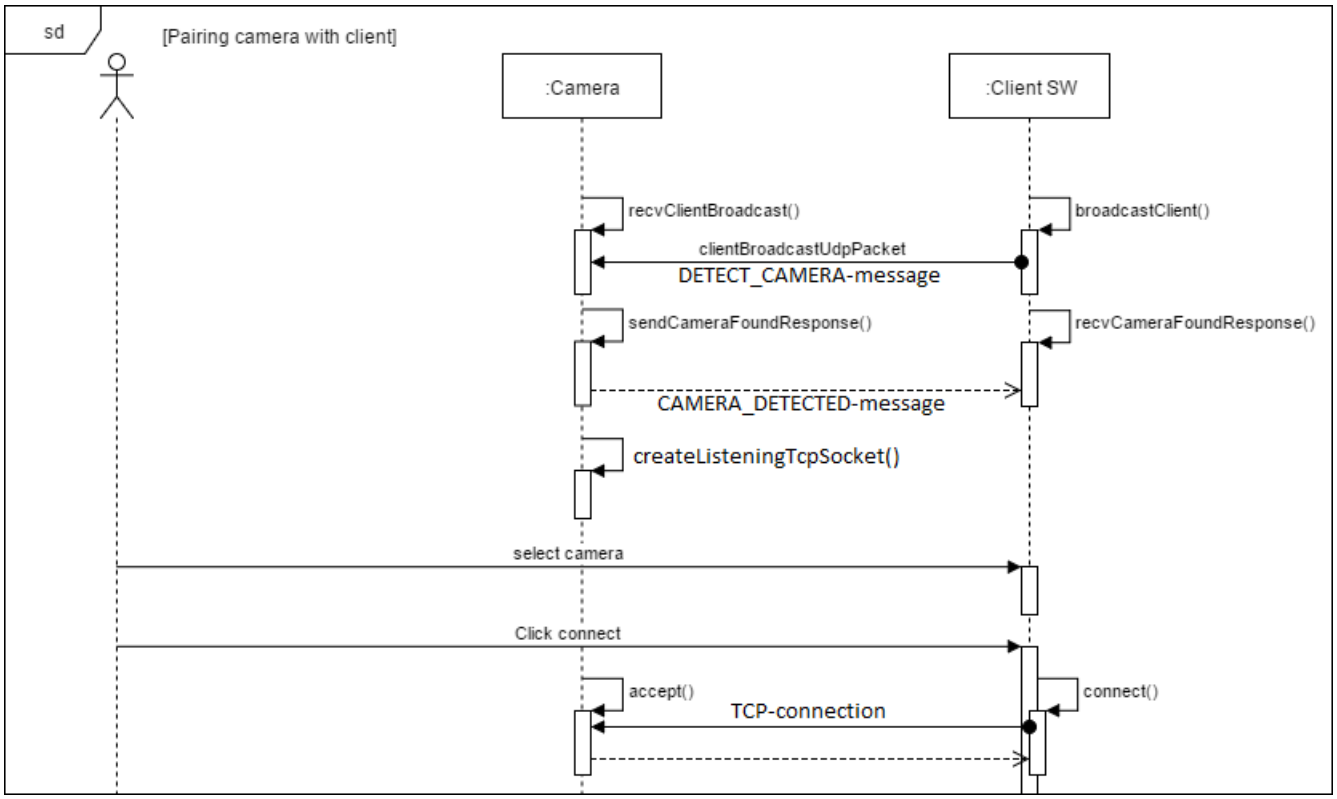

FIGURE 4 PAIRING CAMERA WITH CLIENT

All data fields in the UDP messages should be little-endian when sent over the network. Camera discovery UDP packets used in Camera pairing are defined in Table 1.

Command id	Type	Parameters	Description
DETECT_CAMERA (0x16AC3000)	Request	NULL	Client software broadcasts request as a UDP packet to detect cameras in the network
CAMERA_DETECTED (0x16AC3001)	Response	uint32_t interfaceLevel char macAddress[20] uint32_t cameraReserved uint32_t cameraCustomization char serialNumber[20]	Camera sends response to Client.

TABLE 1 CAMERA DISCOVERY UDP PACKETS

Parameters in CAMERA_DETECTED response are described in Table 2.

Parameter	Description
interfaceLevel	Level of interface between Camera and Client software. ***For Pictor Prestige this field is set to 2.***
macAddress	Mac-address of the Camera. Null-terminated ASCII. Example= "60-64-05-ab-73-7d"
cameraReserved	Flag indicating if Camera is connected to client. If 0, camera is free for new connection. If 1, Camera is connected to Client and new connection cannot be made before last one is terminated.
cameraCustomization	Customization number of Camera. ***Value is 14 for Pictor Prestige.***
serialNumber	Serial-number of the Camera. Null-terminated ASCII.

TABLE 2 CAMERA_DETECTED RESPONSE PARAMETERS

TCP-dump of detection messages between Client and Camera can be seen in Table 3 and Table 4.

BCAST detection packet 09:03:03.258358 IP (tos 0x0, ttl 64, id 15450, offset 0, flags [DF], proto UDP (17), length 32) 10.0.1.170.58672 > 10.0.1.255.3000: [bad udp cksum 0x17c6 -> 0x4afe!] UDP, length 4 0x0000: 4500 0020 3c5a 4000 4011 e6ca 0a00 01aa E...<Z@.@..... 0x0010: 0a00 01ff e530 0bb8 000c 17c6 0030 ac16 .....0.....0..	DETECT_CAMERA

TABLE 3 TCP-DUMP OF DETECT-CAMERA MESSAGE SENT BY CLIENT

REPLY packet 09:03:04.173027 IP (tos 0x0, ttl 128, id 46415, offset 0, flags [none], proto UDP (17), length 84) 10.0.0.142.3000 > 10.0.1.170.58672: [udp sum ok] UDP, length 56 0x0000: 4500 0054 b54f 0000 8011 6f12 0a00 008e E..T.O....o..... 0x0010: 0a00 01aa 0bb8 e530 0040 b505 0130 ac16 .....0.@...0.. 0x0020: 0200 0000 3630 2d36 342d 3035 2d61 622d ....60-64-05-ab- 0x0030: 3733 2d37 6400 0000 0000 0000 0e00 0000 73-7d..... 0x0040: 3338 3138 3538 3130 3330 3136 3700 0000 3818581030167... 0x0050: 0000 0000	CAMERA_DETECTED interfaceLevel, macAddress cameraReserved, cameraCustomization serialNumber

TABLE 4 TCP-DUMP OF CAMERA REPLY TO DETECT-CAMERA MESSAGE

2.3.1 Interface level and compatibility

There are different versions of the communication protocol between Camera and Client. There are also several customization-versions. Based on these two, Client can decide which Cameras to communicate with. Client can for example communicate only with Pictor Prestige -Cameras because they have specific cameraCustomization value in CAMERA_DETECTED -response and specific interfaceLevel (see Table 2).

2.4 Messages

Communication between Camera and Client follows client-server architecture. Camera works as a server, and Client sends requests to Camera. Client can also subscribe to events. Every message described in this chapter is transferred via TCP-socket. Only Camera discovery-messages described in chapter 2.3 are transferred via UDP-socket.

Requests are sent from Client to Camera.

Responses are sent from Camera to Client.

Events are sent from Camera to Client. Events are sent only if Client has subscribed to events (subscribe-request has been sent by Client and responded to by Camera). With events Camera can notify the Client about filesystem changes (files added/deleted). Events are not responded to by Client.

All characters in the requests must be in ascii except for contents of worklist and exif-data in images which may be UTF-8. Character arrays are NULL-terminated.

Messages are transferred via TCP-socket, and message data is divided into message header and payload. For message structure see Table 5. For definition of fields in message header see Table 6.

MESSAGE HEADER *- Same structure for all messages.* *- Size 12 bytes.* Typedef_packed struct messageHeader { **uint32_t cmdId;** **uint32_t code;** **uint32_t seqID;** } messageHeader_t;
PAYLOAD *- Size depends on cmdId and code.* *- Size of payload ≥0 bytes.* *- See chapter 2.4.2 for structure of payload.*

TABLE 5 ENCAPSULATION OF DATA IN MESSAGES BETWEEN CAMERA AND CLIENT.

The protocol for receiving messages is described in Figure 5.

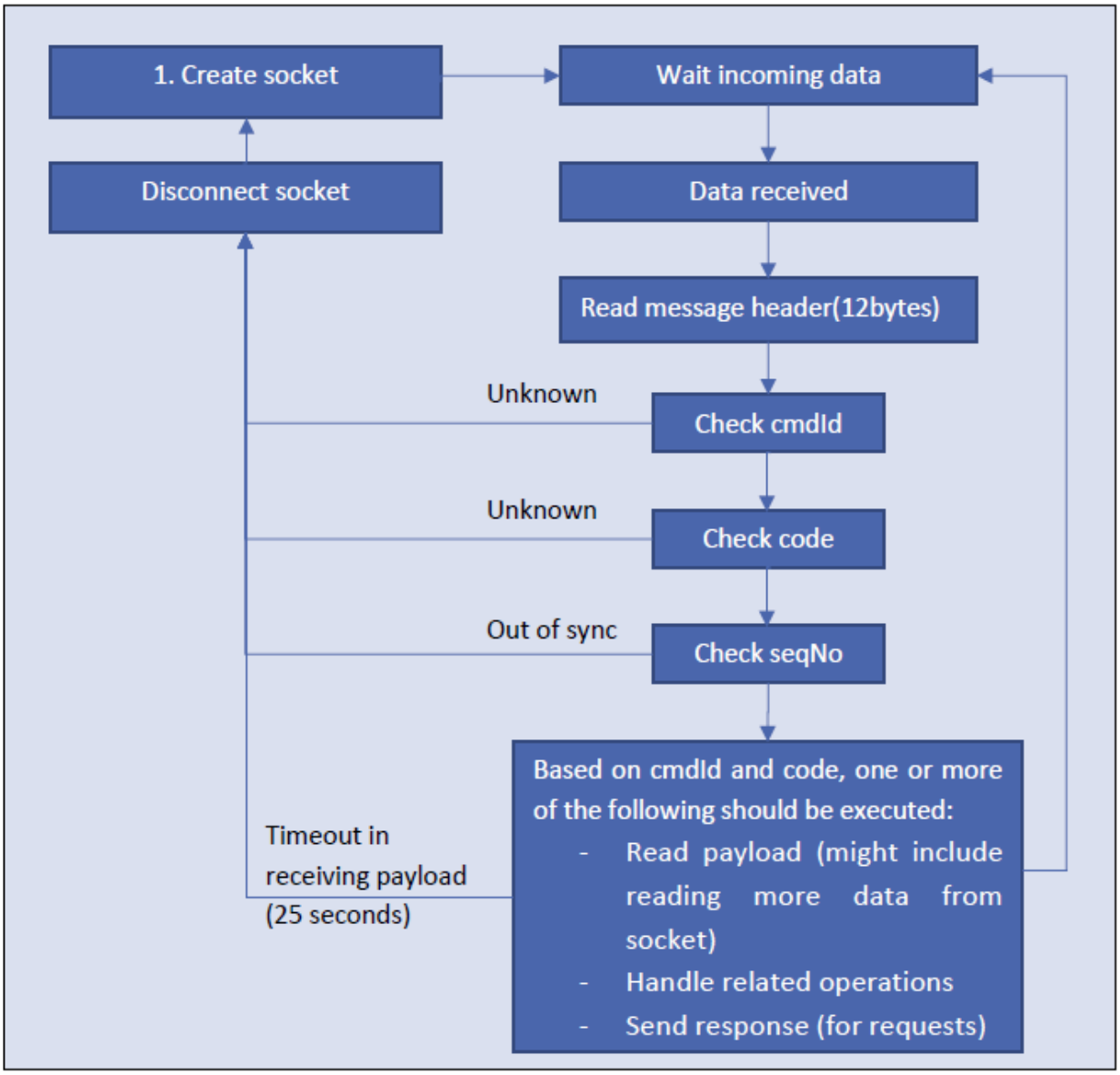

FIGURE 5 RECEIVING MESSAGES (REQUEST, RESPONSE AND EVENT -MESSAGES).

Sequence diagram for messaging between Client and Camera is shown in Figure 6.

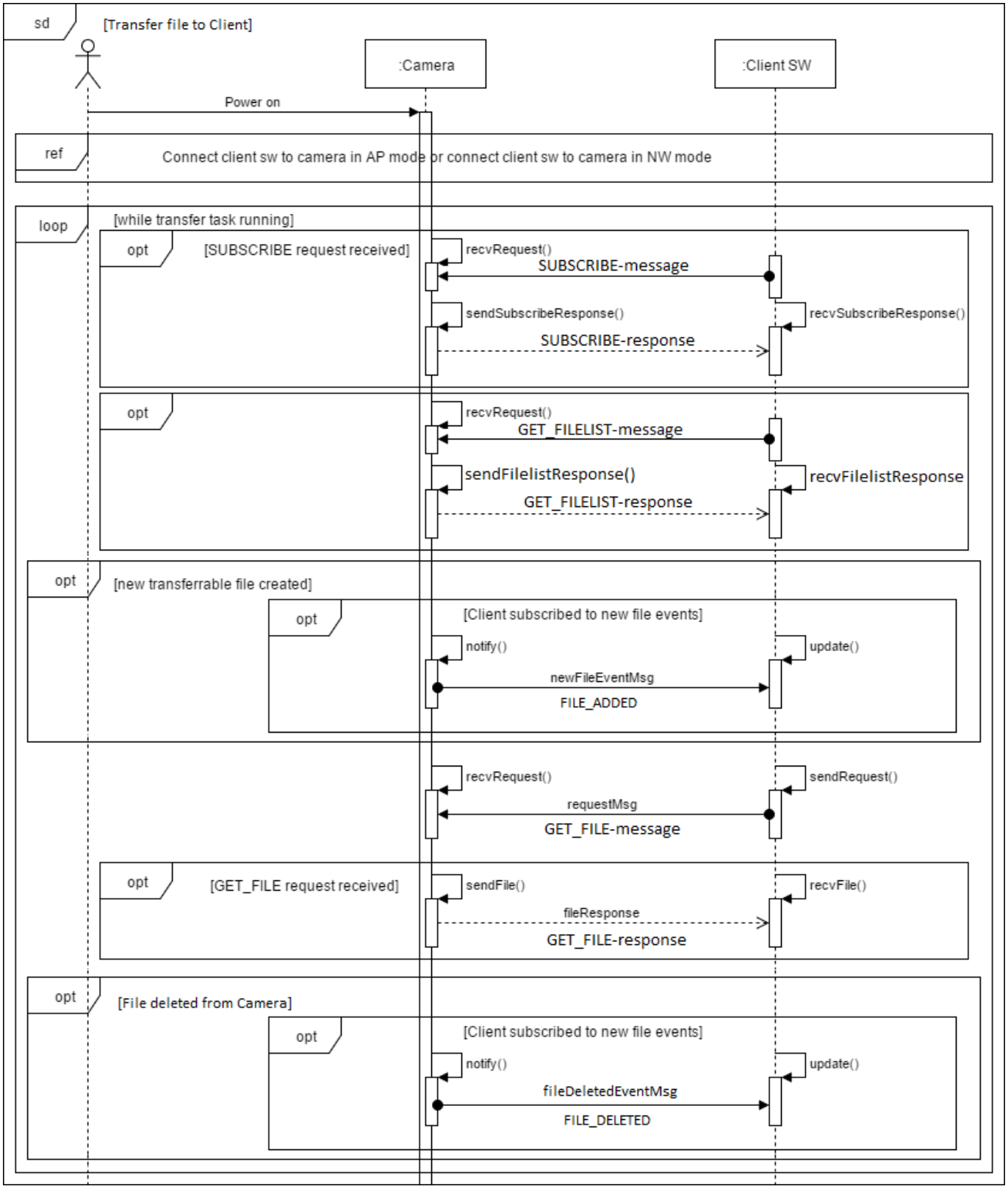

FIGURE 6 FILE EVENTS AND IMAGE TRANSFER. NOTE: SUBSCRIBE-REQUEST HAS TO BE RECEIVED AND ACCEPTED BY CAMERA BEFORE FILE-EVENTS ARE SENT TO CLIENT. WHEN ONE IMAGE IS ADDED OR DELETED FROM CAMERA, EVENT IS SENT WITH INFORMATION ABOUT THAT SPECIFIC FILE SO CLIENT CAN ADD/DELETE IT FROM ITS FILELIST.

2.4.1 Message header

See Table 6 for explanation of fields in message header.

Field	Description
uint32_t cmdId	Command id. Is same for request and response (response for a certain request contains the same cmdId)
uint32_t code	This field can have four possible values: CODE_REQUEST, CODE_EVENT, CODE_OK, CODE_FAIL (these enumerations are defined in chapter 2.5). - CODE_REQUEST means that the message is a request (from Client). - CODE_EVENT means that the message is an event (from Camera). - CODE_OK means that the message is a response for a request, and it was processed successfully. - CODE_FAIL means that the message is a response for a request, and it was processed unsuccessfully. If code is CODE_FAIL, payload contains error-message (see Table 7 for contents of payload).
uint32_t seqId	Sequence number of the message. Created by the creator of the request/event. **Response contains the same seqId-value.** Request from Client to Camera: Add session-unique seqID to a request (for example counter value of sent requests). After receiving response from Camera, this value can be used to make sure the received response is for the correct request sent to Camera. Event from Camera to Client: Counter value of sent events. If Client misses one seqID (ie. previous seqID was 4 and next seqID is 6), it knows that file-events are not in sync and the full filelist must be requested from Camera.

TABLE 6 EXPLANATION OF MESSAGE HEADER FIELDS. MESSAGE HEADER-STRUCTURE IS SAME FOR ALL MESSAGES (REQUEST-MESSAGE, RESPONSE-MESSAGE, EVENT-MESSAGE).

2.4.2 Payload

It is possible for messages to include payload after the message header. The structure of the payload is defined based on cmdId and code. In this chapter the payload structures for different messages are defined.

All structures are defined with “packed”-keyword.

See Table 7 for structure of payload for different messages.

cmdId	Code	Payload
SUBSCRIBE (0x16AC4003) *- Client requests filelist-events from Camera*	CODE_REQUEST	No payload
SUBSCRIBE (0x16AC4003) *- Camera response to SUBSCRIBE*	CODE_OK	No payload
UNSUBSCRIBE (0x16AC4009) *- Client cancels subscription to filelist-events*	CODE_REQUEST	No payload
UNSUBSCRIBE (0x16AC4009) *- Camera response to UNSUBSCRIBE*	CODE_OK	No payload
GET_FILE (0x16AC4004) *- Client requests a file from Camera*	CODE_REQUEST	char filepath[28]; // Path of file in Camera SD-card **EXAMPLE:** *“\DCIM\P0001\IM0001.JPG”*
GET_FILE (0x16AC4004) *- Camera response to GET_FILE*	CODE_OK	typedef __packed struct getFileResponse { fileInfo_t fileInfo; // Info of file. See FILE_ADDED-cmdId. uint8_t byteArray[]; // Filedata } getFileResponse_t;
POST_FILE (0x16AC4005) *- Client sends file to Camera (patlist.txt)*	CODE_REQUEST	typedef __packed struct postFile { uint32_t filetype; // Type of file. See chapter 2.5. uint32_t bytes; // Amount of bytes in byteArray char filename[12]; // Name of file uint8_t byteArray[]; // Filedata } postFile_t;
POST_FILE (0x16AC4005) *- Camera response to POST_FILE*	CODE_OK	No payload
FILE_ADDED (0x16AC4108) *- Camera sends event to Client, telling that new file has been added (when new image is taken with Camera)*	CODE_EVENT	typedef __packed struct fileInfo { uint32_t filesize; // File-size uint32_t fileType; // Type of file(= 0x2). See chapter 2.5 uint16_t fileDate; // FAT32 timestamp date uint16_t fileTime; // FAT32 timestamp time char filename[28]; // Full path of file in Camera SD-card } fileInfo_t;

FILE_DELETED (0x16AC4107) *- Camera sends event to Client, telling that file has been removed from Camera*	CODE_EVENT	typedef __packed struct fileInfo { uint32_t filesize; // File-size uint32_t fileType; // Type of file. See chapter 2.5. uint16_t fileDate; // FAT32 timestamp date uint16_t fileTime; // FAT32 timestamp time char filename[28]; // Full path of file in Camera SD-card } fileInfo_t;
FILELIST_UPDATED (0x16AC4110) *- Camera sends event to Client, informing that filelist should be requested from Camera (this event is sent when Camera is lifted from Cradle, image memory has been erased, or patient folder has been deleted)*	CODE_EVENT	No payload
GET_FILELIST (0x16AC4007) *- Client requests filelist from Camera*	CODE_REQUEST	char filepath[28] // Path in Camera SD-card. All // subfolders will be included in filelist **NOTE:** When asking filelist for all folders, filepath should be "*DCIM*". This is the root directory for all images.
GET_FILELIST (0x16AC4007) *- Camera response to GET_FILELIST*	CODE_OK	uint32_t count; // Count of files in list fileInfo files[]; // List of files typedef __packed struct fileInfo { uint32_t filesize; // File-size uint32_t fileType; // Type of file. See chapter 2.5. uint16_t fileDate; // FAT32 timestamp date uint16_t fileTime; // FAT32 timestamp time char filename[28]; // Full path of file in Camera SD-card } fileInfo_t;
GET_CAMERA_STATUS (0x16AC4008) *- Client requests Camera status. Used to get information about Camera.*	CODE_REQUEST	No payload
GET_CAMERA_STATUS (0x16AC4008) *- Camera response to GET_CAMERA_STATUS*	CODE_OK	Typedef __packed struct cameraStatus { uint32_t clientSubscribed; // 1 if Client is subscribed. char cameraSwVersion[16]; // SW version of Camera char wifiSwVersion[16]; // SW version of wifi-module } cameraStatus_t;

PING_CAMERA (0x16AC4010) *Can be used to check if Camera is online and responsive.*	CODE_REQUEST	No payload
PING_CAMERA (0x16AC4010) *- Camera response to* PING_CAMERA	CODE_OK	No payload
Applicable to all cmdId:s	CODE_FAIL	typedef __packed struct messageFail { uint32_t errCode; // Error-code. See chapter 2.5 for // possible values char errMsg[64]; // Error-message } messageFail_t;

TABLE 7 PAYLOAD STRUCTURES

TCP-dump of GET_FILELIST-request can be seen in Table 8 and reply to GET_FILELIST-request in Table 9.

```
GET_FILELIST request
09:14:42.399371 IP (tos 0x0, ttl 64, id 38701, offset 0, flags [DF],
proto TCP (6), length 116)
    10.0.1.170.46142 > 10.0.0.142.8000: Flags [P.], cksum 0x169e
(incorrect -> 0x5305), seq 1:77, ack 1, win 29200, length 76
  0x0000:  4500 0074 972d 4000 4006 8d1f 0a00 01aa  E..t.-@.@.......
  0x0010:  0a00 008e b43e 1f40 563e 55c8 faaf 09b3  .....>.@V>U.....
  0x0020:  5018 7210 169e 0000 0740 ac16 0350 ac16  P.r.....@...P..
  0x0030:  0100 0000 5c44 4349 4d00 0000 0000 0000  ....\DCIM.......
  0x0040:  0000 0000 0000 0000 0000 0000 0000 0000  ................
  0x0050:  0000 0000 0000 0000 0000 0000 0000 0000  ................
  0x0060:  0000 0000 0000 0000 0000 0000 0000 0000  ................
  0x0070:  0000 0000
```

```
uint32_t cmdId, uint32_t code
uint32_t seqID, char filepath[64]
```

TABLE 8 TCP-DUMP OF GET_FILELIST-REQUEST SENT BY CLIENT

```
GET_FILELIST response
09:14:43.009242 IP (tos 0x0, ttl 128, id 25408, offset 0, flags [none],
proto TCP (6), length 336)
    10.0.0.142.8000 > 10.0.1.170.46142: Flags [P.], cksum 0x215f
(correct), seq 1:297, ack 77, win 33580, length 296
  0x0000:  4500 0150 6340 0000 8006 c030 0a00 008e  E..Pc@.....0....
  0x0010:  0a00 01aa 1f40 b43e faaf 09b3 563e 5614  .....@.>....V>V.
  0x0020:  5018 832c 215f 0000 0740 ac16 0150 ac16  P..,!_...@...Q..
  0x0030:  0100 0000 [illegible] 0000 0000 1000 0000  ................
  0x0040:  2128 d408 5c64 6369 6d5c 5030 3030 3100  !(. \dcim\P0001.
  0x0050:  0000 0000 0000 0000 0000 0000 0000 0000  ................
  0x0060:  d052 1200 2000 0000 2128 d508 5c64 6369  .R.....!(..\dci
  0x0070:  6d5c 5030 3030 315c 494d 3030 3031 4559  m\P0001\IM0001EY
  0x0080:  2e4a 5047 0000 0000 2051 1200 2000 0000  .JPG..... Q.....
  0x0090:  2128 fa08 5c64 6369 6d5c 5030 3030 315c  !(..\dcim\P0001\
  0x00a0:  494d 3030 3032 4559 2e4a 5047 0000 0000  IM0002EY.JPG....
  0x00b0:  3046 1200 2000 0000 2128 ca10 5c64 6369  0F..... !(..\dci
  0x00c0:  6d5c 5030 3030 315c 494d 3030 3033 4559  m\P0001\IM0003EY
  0x00d0:  2e4a 5047 0000 0000 e049 1200 2000 0000  .JPG.....I.....
  0x00e0:  2128 fa10 5c64 6369 6d5c 5030 3030 315c  !(..\dcim\P0001\
  0x00f0:  494d 3030 3034 4559 2e4a 5047 0000 0000  IM0004EY.JPG....
  0x0100:  b04d 1200 2000 0000 2128 941b 5c64 6369  .M..... !(..\dci
  0x0110:  6d5c 5030 3030 315c 494d 3030 3035 4559  m\P0001\IM0005EY
  0x0120:  2e4a 5047 0000 0000 c030 1200 2000 0000  .JPG.....0.....
  0x0130:  2128 cd2e 5c64 6369 6d5c 5030 3030 315c  !(..\dcim\P0001\
  0x0140:  494d 3030 3036 4559 2e4a 5047 0000 0000  IM0006EY.JPG....
```

```
uint32_t cmdId, uint32_t code
uint32_t seqID, uint32_t count,
fileinfo_t fileInfo1,

fileinfo_t fileInfo2

[illegible],

fileinfo_t fileInfo4

[illegible],

[illegible]

fileinfo_t fileInfo7,
```

TABLE 9 TCP-DUMP OF GET_FILELIST-RESPONSE SENT BY CLIENT

2.5 Enumerations

Enumerations are defined in this chapter.

Command ID:s (“CmdId” in message header):

```
CMDID_SUBSCRIBE              = 0x16AC4003;
CMDID_UNSUBSCRIBE            = 0x16AC4009;
CMDID_GET_FILE               = 0x16AC4004;
CMDID_POST_FILE              = 0x16AC4005;
CMDID_GET_FILELIST           = 0x16AC4007;
CMDID_GET_CAMERA_STATUS  =  0x16AC4008;
CMDID_PING_CAMERA            = 0x16AC4010;
CMDID_FILE_ADDED             = 0x16AC4108;
CMDID_FILE_DELETED           = 0x16AC4107;
CMDID_FILELIST_UPDATED       = 0x16AC4110;
```

Message header codes (“code” in message header):

```
CODE_OK          = 0x16AC5001;
CODE_FAIL        = 0x16AC5002;
CODE_REQUEST  =  0x16AC5003;
CODE_EVENT       = 0x16AC5004;
```

Error codes (“errCode” in CODE_FAIL response):

```
ERR_UNKNOWN_CMDID    =    0x16AC6001,
ERR_STATE_CRADLE          = 0x16AC6002,
ERR_INVALID_REQUEST       = 0x16AC6003,
ERR_CAMERA_ERROR          = 0x16AC6004,
ERR_FILE_DOES_NOT_EXIST  =  0x16AC6005,
ERR_INVALID_FILE          = 0x16AC6006,
ERR_UNKNOWN_TYPE          = 0x16AC6007,
ERR_TRANSFER              = 0x16AC6008,
```

Type of files (fileType-parameter in payload-structures):

```
ReadOnly    = 0x1;
Hidden      = 0x2;
System      = 0x4;
Volume      = 0x8;
Directory   = 0x10;
File        = 0x20;
UNKNOWN = 0x40,
```

2.6 Uploading a file from Client to Camera

In this version of API, only file that can be uploaded to Camera is patlist.txt containing list of Patient ID and name -combinations (worklist). File is saved to root-directory of SD-card and updated from there to Camera. Old patient-list is overwritten.

The patlist.txt-file should follow format (where <CR> is carriage return (0x0D) and <LF> is line feed (0x0A)):

```
<CR><LF>
patientId<CR><LF>
patientName<CR><LF>
<CR><LF>
patientId<CR><LF>
patientName <CR><LF>
<CR><LF>
patientId<CR><LF>
patientName <CR><LF>
```

Maximum length of line in patient list is 32 UTF-8 characters, excluding the line break in the end of the field. Maximum number of patients in the list is 500. Following characters are supported:

```
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ(),.:-_@/
ÄäÉéÖöÜüß
åÅäÄöÖ
ÀàÂâÆæÇçÈèÉéÊêËëÎîÏïÔôÙùÛûÜü
ÀàÂâÁáÆæÇçÈèÉéÊêËëÌìÍíÎîÏïÒòÓóÔôÕõÙùÚúÛûÜüÑñ
ÀàÂâÆæÇçÈèÉéÊêËëÌìÍíÎîÏïÒòÓóÔôÙùÚúÛûÜüÑñ
```

If the character is not in the supported list, '?' is shown in Camera GUI in place of it.

Sequence diagram of file upload from Client to Camera is shown in Figure 7.

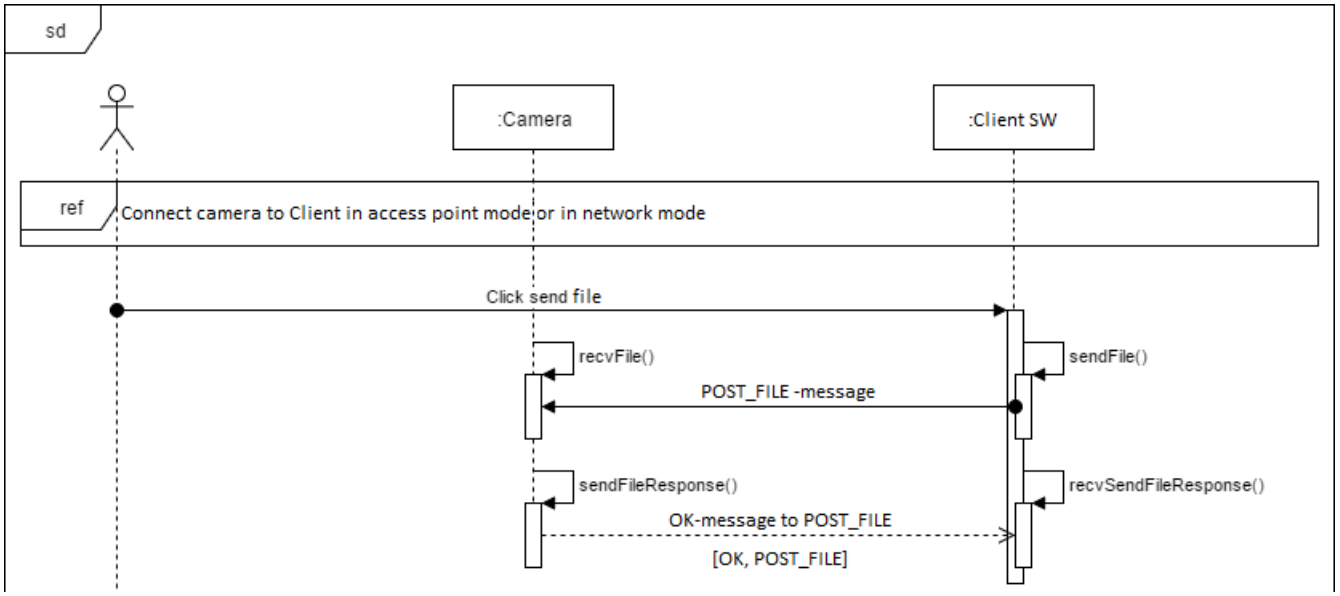

FIGURE 7 TRANSFER WORKLIST TO CAMERA FROM CLIENT.

2.7 Security

In current version of the API, TCP-messages are not encrypted. For better security, it is advisable to use Camera Access Point -mode to create connection between Client and Camera (WPA2 security protocol is used in AP-mode and no other devices cannot connect to the same network without password). In Camera network mode, a trusted private access point should be used. Public access points should not be used forsecurity reasons.